HW4 Report

1. Part 2

1.1 The Lifecycle of a Cloud Task

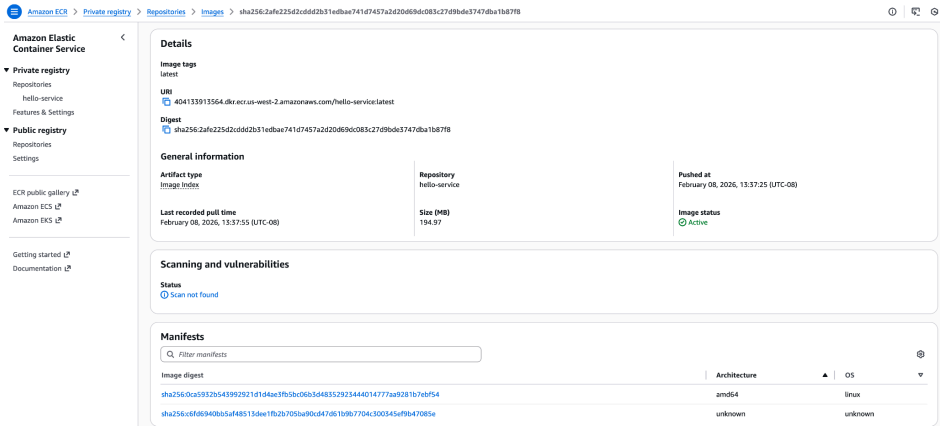
In this phase, I established the "Build-Push-Launch" workflow required for modern cloud-native applications:

1. **Docker image:** build a new docker image based on hw2.
2. **ECR (Elastic Container Registry):** Managed image versions and permissions.
3. **ECS (Elastic Container Service):** Deployed tasks using the serverless Fargate launch type.

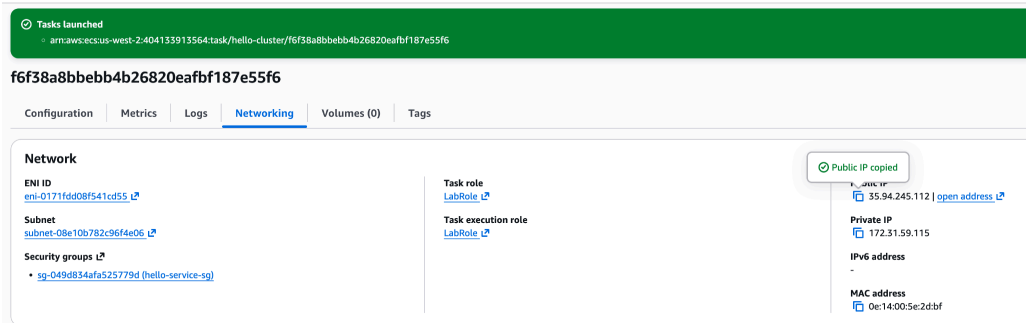
Here are the 7 steps

1. Configure AWS.
2. Go to Amazon ECR to create a repository, then copy the repository URI.
3. Push Docker image to ECR.

```
molly@mollys-MacBook-Pro hw4 % molly@mollys-MacBook-Pro hw4 % aws ecr
 list-images \
  --repository-name hello-service \
  --region us-west-2 \
  --query 'imageIds[*].imageTag' \
  --output table
------------
|ListImages|
+----------+
|  latest  |
+----------+
```



4. Create an ECS Cluster.
5. Create task definition.
6. Run a task.

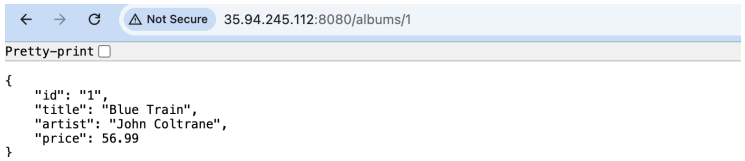


7. Getting the Public IP and Testing.

```
molly@mollys-MacBook-Pro hw4 % curl http://35.94.245.112:8080/albums

[
    {
        "id": "1",
        "title": "Blue Train",
        "artist": "John Coltrane",
        "price": 56.99
    },
    {
        "id": "2",
        "title": "Jeru",
        "artist": "Gerry Mulligan",
        "price": 17.99
    },
    {
        "id": "3",
        "title": "Sarah Vaughan and Clifford Brown",
        "artist": "Sarah Vaughan",
        "price": 39.99
    }
]%
molly@mollys-MacBook-Pro hw4 % curl http://35.94.245.112:8080/albums/
1

{
    "id": "1",
    "title": "Blue Train",
    "artist": "John Coltrane",
    "price": 56.99
}%
```



1.2 Technical Concept Exploration

Difference between EC2 and ECS:

- **EC2 (Elastic Compute Cloud):** Provides raw Virtual Machines. You are responsible for managing the OS, patching, and installing Docker yourself. It offers the most control but the highest management overhead.
- **ECS (Elastic Container Service):** An orchestration service. With **Fargate**, it is serverless, meaning AWS manages the underlying servers. You only provide the container image and resource requirements.

VPC and Subnets:

- **VPC (Virtual Private Cloud):** A logically isolated virtual network in AWS. It defines the

networking environment for your resources.

- **Subnet:** A range of IP addresses within a VPC. It allows you to group resources based on security and operational needs.
- **Accessing the Default VPC:** AWS provides a default VPC in every region with public subnets. I accessed it by assigning a **Public IP** to my Fargate tasks and configuring **Security Groups** (virtual firewalls) to allow inbound traffic on port 8080.

TCP vs. UDP:

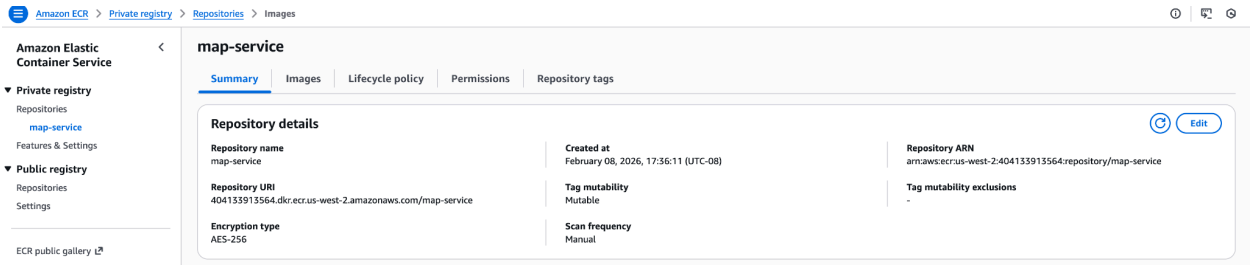
- **TCP (Transmission Control Protocol):** Connection-oriented and reliable. It ensures all packets arrive in the correct order. This is essential for our MapReduce project because a single missing word count would invalidate the results.
- **UDP (User Datagram Protocol):** Connectionless and faster, but does not guarantee delivery. It is used for streaming or gaming where speed is prioritized over 100% data integrity.

Controlling Resource Allocation: Resources are controlled in the **Task Definition**:

- **Task Level:** Sets the total CPU and Memory for all containers in the task.
- **Container Level:** Defines **Hard Limits** (task is killed if exceeded) and **Soft Limits** (guaranteed reservation). This ensures one container doesn't consume all resources on a shared host.

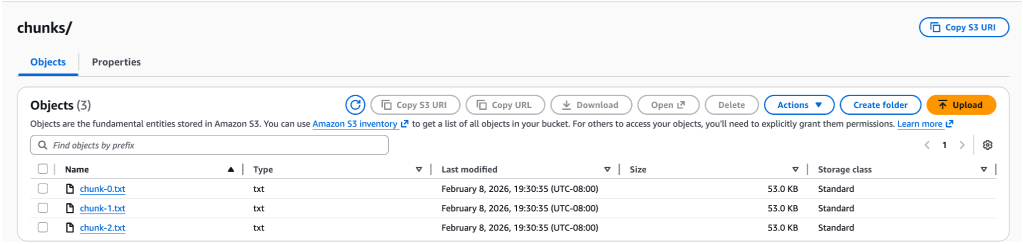
2. Part 3: Distributed MapReduce Implementation

2.1 Implementation Logic

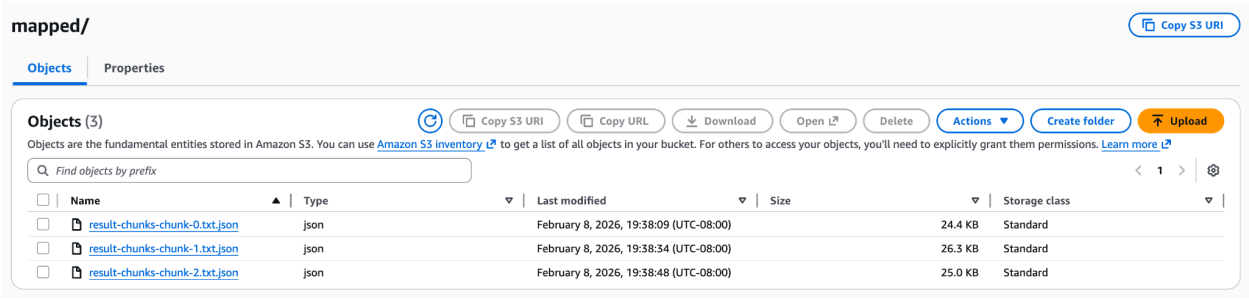


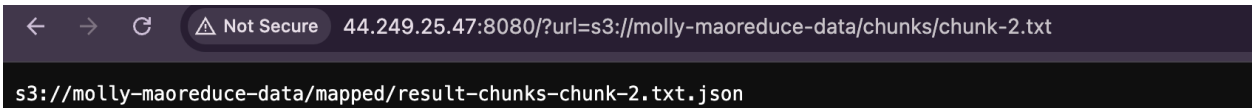
The project followed a **"Single Binary, Multiple Roles"** pattern. I implemented three distinct functions within one Go program, triggered by the APP_ROLE environment variable:

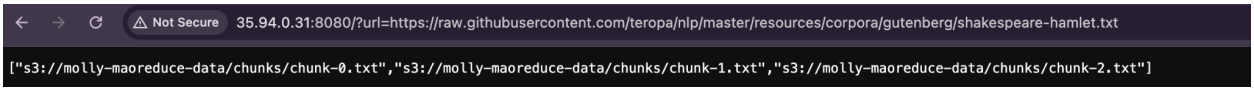
1. **Splitter:** Reads the source text from a URL, calculates split points based on character count while respecting newline boundaries, and uploads three .txt chunks to S3.



2. **Mapper:** Triggered via a URL parameter, it downloads a specific chunk from S3, tokenizes the text, standardizes it (lowercase/no punctuation), and uploads a JSON map of word counts.

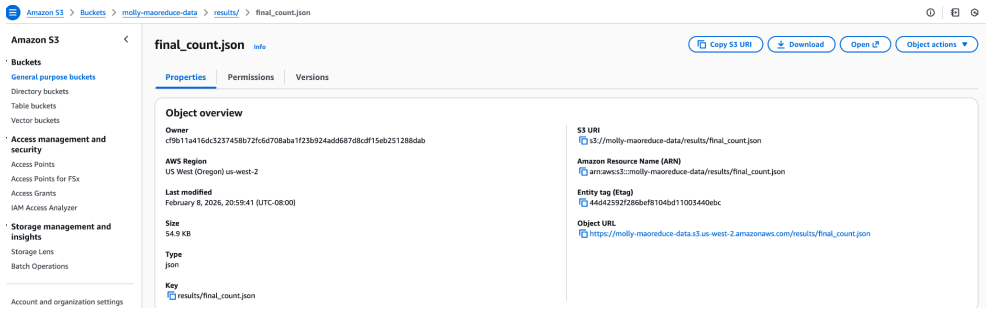






3. **Reducer:** Receives multiple Mapper URLs, downloads all JSON results, aggregates the

counts into a single master map, and saves the final final_count.json.



2.2 Challenges and Debugging

During implementation, I encountered and resolved several critical issues:

- **ECR Tag Immutability:** Overwriting the :latest tag was initially blocked. I had to change the repository settings to **Mutable** to allow for iterative code updates.
- **Silent Failures (Empty Results):** Early runs produced an empty {} result. This was due to the Reducer attempting to process S3 links before the Mappers had finished writing or due to incorrect URL parameter parsing. I added **Robust Logging** and **Error Checks** to ensure the Reducer only proceeds if data is successfully downloaded.
- **Manual Orchestration:** Coordinating the "relay race" by manually passing URLs between browser tabs was the most challenging part, highlighting the value of automation tools like Terraform or Step Functions.

2.3 Verification

I verified the final output using a Python verification script. The counts (e.g., hamlet: 100) matched the source text exactly, proving the distributed logic is sound and the data remained consistent across the S3 hand-offs.

```
(.venv) molly@mollys-MacBook-Pro part3 % python result-verification.py
Downloading file from source for ground truth verification...
URL: https://raw.githubusercontent.com/teropa/nlp/master/resources/corpora/gutenberg/shakespeare-hamlet.txt
Local verification complete! Total word count: 30266

Word         | Ground Truth    | Your System     | Status
-------------------------------------------------------------
hamlet       | 100             | 100             | ✅ Match
hand         | 19              | 19              | ✅ Match
lord         | 211             | 211             | ✅ Match
queen        | 8               | 8               | ✅ Match
horatio      | 40              | 40              | ✅ Match

Conclusion: Your system's results match the source file perfectly! The counts are small because the source file itself is small.
```

2.4 Performance Analysis（Doing）

(Placeholder for charts and performance data)

- **Baseline (Sequential) Time:** [X] seconds
- **Distributed (Split-Map-Reduce) Total Time:** [Y] seconds

- **Analysis:** [Insert analysis regarding the trade-off between parallel computation and S3 I/O latency here]